Alumnos:

Marco Antonio Aguilar Castillo

Cleber Antonio Bolaños Moreno

Osbaldo Alvarez Martinez

Kevin Ricardo Simon Alfaro

Samuel Jonathan Trujillo Bolaños

Docente:

José Miguel Carrera Pacheco

8°B

Proyecto: Conecta-Tehuacán

INVESTIGACION:

S - 11. Manejo de multisesiones y persistencia

Manejo de multisesiones y persistencia

¿Qué es una multisesión?

Una **multisesión** se refiere a la capacidad de un sistema para permitir que un mismo usuario mantenga **múltiples sesiones activas simultáneamente**, cada una identificada por un token único (por ejemplo, cookies de sesión o JSON Web Tokens).

Desde el punto de vista de arquitectura, una sesión representa un **estado persistente temporal** entre cliente y servidor, utilizado para mantener autenticación y contexto (Fielding, 2000).

En aplicaciones modernas:

- En arquitecturas **stateful**, la sesión se guarda en el servidor (memoria, Redis, base de datos).
- En arquitecturas **stateless**, se utilizan tokens como **JWT**, donde el estado viaja en cada petición.

¿Qué problemas surgen con sesiones concurrentes?

1. Vulnerabilidades de seguridad

Las sesiones múltiples incrementan la superficie de ataque, especialmente en casos de:

- **Session hijacking**
- **Session fixation**

OWASP (2021) señala que el manejo inadecuado de sesiones es una de las principales causas de accesos no autorizados.

2. Condiciones de carrera (Race Conditions)

Cuando múltiples sesiones modifican datos simultáneamente:

- Se generan inconsistencias
- Se pueden sobrescribir cambios sin control

Esto es común en sistemas sin mecanismos de **control de concurrencia**.

3. Sobrecarga del sistema

- Cada sesión consume recursos (memoria, conexiones)
- En sistemas distribuidos, implica sincronización entre nodos

4. Problemas de control de autenticación

- No se puede limitar fácilmente el número de sesiones
- Difícil invalidar sesiones activas en todos los dispositivos

¿Cómo se invalida una sesión?

1. Eliminación directa (Stateful)

- Se elimina el registro de sesión del almacenamiento
- Ejemplo: eliminar clave en Redis

2. Expiración (Timeout)

- Basada en:
    - Tiempo de inactividad (idle timeout)
    - Tiempo absoluto (absolute timeout)

3. Blacklisting de tokens (JWT)

- Se mantiene una lista de tokens inválidos
- Se usa cuando no hay estado en servidor

4. Rotación de credenciales

- Cambio de contraseña invalida sesiones
- Regeneración de tokens

5. Revocación centralizada

- Sistemas modernos usan servicios de autenticación (ej: OAuth2)

- Permiten invalidar sesiones globalmente

¿Qué riesgos existen si no se controlan correctamente?

1. Session Hijacking

Ataques donde un tercero obtiene un identificador de sesión válido.

2. Session Fixation

El atacante fuerza a la víctima a usar un ID de sesión conocido.

3. Replay Attacks

Reutilización de tokens interceptados.

4. Persistencia indebida

Sesiones que permanecen activas indefinidamente.

5. Escalada de privilegios

Uso de sesiones antiguas con permisos elevados.

Persistencia de sesiones

La persistencia define **dónde y cómo se almacenan las sesiones**:

Tipos comunes:

- **Memoria local**
  - Rápido pero no escalable
- **Redis**
  - Alta velocidad
  - Soporte para TTL (muy usado en backend)
- **Base de datos**
  - Persistente pero más lento

- **JWT (stateless)**
    - No requiere almacenamiento en servidor
    - Mayor complejidad para invalidación

Buenas prácticas

- Limitar número de sesiones simultáneas por usuario
- Usar expiración corta con refresh tokens
- Implementar HTTPS siempre
- Regenerar ID de sesión tras login
- Monitorear sesiones activas
- Usar almacenamiento seguro (ej: Redis)

Referencias

- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. University of California, Irvine.
- OWASP Foundation. (2021). *OWASP Top 10: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/
- Hardt, D. (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. https://doi.org/10.17487/RFC7519
- Redis Ltd. (2023). *Redis documentation*. https://redis.io/docs/
- Stallings, W. (2017). *Cryptography and network security: Principles and practice* (7th ed.). Pearson.